MATERIAL DE ANALIZA / DISCUTIE COMUNITARA

Increderea in agentii AI

Organizarea conversatiei, firul narativ, ideile principale si un raspuns propus pentru continuarea discutiei.



24	8	5
INTERVENTII	PARTICIPANTI	ETAPE TEMATICE

INPUT

Context si cerinte

Textul de mai jos reprezinta o colectie de pe facebook, in ordinea in care au fost postate mesajele si raspunsurile lor.

01 Pasul1: Vreau sa oraganizezi conversatia intr-o ordine citibila. Cine si ce a zis, in ordine.

02 Pasul 2: Incearca sa intelegi firul narativ si sa vezi despre ce se vorbea

03 Pasul 3: Fa rezumatul discutiilor si extrage ideile principale.

04 Pasul 4: Daca ar fi sa participi activ in acea conversatie, ce ai raspunde?

Material analizat: conversatie Facebook cu postare initiala si raspunsuri in ordinea furnizata. Transcrierea completa, organizata dupa autor si etape, este inclusa la finalul documentului.

OUTPUT

Analiza discutiei

Concluzia principala: discutia a evoluat de la intrebarea simpla „Putem avea incredere in agentii AI?” la construirea unei metode reproductibile pentru evaluarea agentilor, orchestratoarelor, dovezilor si conditiilor in care un sistem AI poate executa actiuni autonome.

24 de interventii, inclusiv postarea initiala, de la **8 participanti distincti**. Ordinea este cea din fisierul-sursa, nu o cronologie confirmata prin timestamp-uri. Autorul postarii initiale pare sa fie Sorin Balan, conform raspunsurilor celorlalti.

01 / Conversatia reorganizata cronologic

Sinopsisul fiecarei interventii, in ordinea aparitiei

ETAPA I. DE UNDE A PORNIT DISCUTIA

NR.	PARTICIPANT	CONTRIBUTIE
00	**Sorin Balan**	Deschide discutia pornind de la dilema lui Ovidiu. Invoca diferentele dintre software determinist si agenti AI, metodele de evaluare si human-in-the-loop. Cere un material comun despre increderea in agenti.
01	**Paul Stoica**	Propune monitorizarea atat a rezultatului, cat si a executiei. Considera logurile si monitorizarea obligatorii.
02	**Adrian Moise**	Explica de ce evaluarea agentilor difera de testarea clasica. Propune capturarea prompturilor, contextului, apelurilor de instrumente, datelor returnate si rezultatului final. Introduce verificarea prin al doilea LLM.
03	**Ovidiu Boticiu**	Clarifica intrebarea initiala. Nu cere agenti infailibili, ci modalitati de a verifica ce au executat si daca rezultatele sunt corecte. Da exemplul agentului meteo care poate raspunde fara instrument.
04	**Cri Stian**	Sustine pozitia lui Ovidiu. Propune teste automate, verificarea datelor si interventia umana pentru operatiunile riscante. Avertizeaza ca verdictul unui alt LLM nu este suficient.

> **Un raspuns plauzibil nu reprezinta dovada unei executii corecte.**

ETAPA II. PERFORMANTA VERSUS VERIFICABILITATE

NR.	PARTICIPANT	CONTRIBUTIE
05	**Daniel Diaconu**	Subliniaza progresul rapid al modelelor, considera sarcinile simple aproape perfect rezolvabile si discuta limitele datelor pentru sarcini complexe; mentioneaza folosirea agentilor in productie.
06	**Cri Stian**	Separa performanta de verificabilitate. Nici cele mai puternice modele nu garanteaza zero erori; cere dovezi, verificari si rollback in productie.
07	**Daniel Diaconu**	Accepta verificarea, dar ridica intrebarea daca oamenii vor mai putea intelege si controla executia agentilor mai complecsi.
08	**Cri Stian**	Propune evaluarea progresiva in medii controlate, teste repetate si cresterea treptata a autonomiei; introduce Trust Score diferentiat pe clase de sarcini.
09	**Daniel Diaconu**	Accepta necesitatea evaluarii progresive si descrie experienta proprie de trial-and-error inclusiv in productie.

De la verificarea fiecarui raspuns la evaluarea sistematica a comportamentului unui agent.

ETAPA III. ARHITECTURA MULTI-AGENT SI ORCHESTRARE

NR.	PARTICIPANT	CONTRIBUTIE
10	**Cri Stian**	Propune agenti specializati pentru cercetare, implementare, testare si verificare, coordonati de orchestrator; separarea memoriei de contextul activ.
11	**Sorin Balan**	Observa asemanari cu arhitectura prezentata de Cyber AI Romania si sustine directia discutiei.
12	**Daniel Diaconu**	Descrie un proiect de memorie comuna pentru Hermes, Claude, ChatGPT, LiteLLM si n8n, cu Forgejo, Obsidian, Basic Memory si acontext.
13	**Cyber AI Romania**	Prezinta un sistem propriu cu sapte agenti specializati, un orchestrator, un command center si modele diferite pe sarcini.
14	**Daniel Diaconu**	Explica dificultatile gestionarii a circa 30 de proiecte, pierderea contextului si interesul pentru exemple open-source.
15	**Ovidiu Boticiu**	Intreaba cine verifica orchestratorul si cere o comparatie obiectiva intre sistemele multi-agent si un singur agent.
16	**Sorin Balan**	Aduce articolul lui Mario Zechner despre agentul minimalist Pi, ca alternativa la cresterea complexitatii.
17	**Cri Stian**	Propune un benchmark cu patru configuratii de la agent generalist la orchestrare dinamica. Sustine teste reproductibile, controlul variabilelor si comparatia cu solutii mai simple.

O arhitectura mai complexa trebuie sa isi demonstreze beneficiile, nu sa le presupuna.

ETAPA IV. COMPETENTA, AUTORITATE SI DOVEZI

NR.	PARTICIPANT	CONTRIBUTIE
18	**Liviu Sapunaru**	Separa competenta demonstrata de autoritatea acordata. Performanta buna nu justifica automat cresterea drepturilor.
19	**Cri Stian**	Formalizeaza Trust Score si Authority Level ca dimensiuni separate; extinderea permisiunilor necesita decizie explicita.
20	**Liviu Sapunaru**	Adauga suficienta dovezilor pentru operatiunea concreta. Verificarea probelor trebuie sa fie o conditie tehnica obligatorie.

> **Un agent competent si autorizat poate totusi sa nu aiba dovezile necesare pentru executie.**

ETAPA V. LITERATURA TRANSFORMATA IN TESTE

NR.	PARTICIPANT	CONTRIBUTIE
21	**Liviu Sapunaru**	Prezinta 20 de pasaje literare transpuse in 12 familii de erori decizionale. Ofera cazul unui agent care primeste dovada, dar decide ca lipseste; arata riscul surselor corelate.
22	**Cri Stian**	Propune evaluarea provenientei surselor, a erorilor corelate si comparatia dintre context comun si surse independente. Cere metodologia si traseul complet.
23	**Liviu Sapunaru**	Detaliaza LAB27 si LAB28, confirma experimente, delimiteaza concluziile care nu au fost demonstrate si propune perechi de teste cu surse comune versus independente.

> **Confirmarile repetate nu sunt automat dovezi independente.**

02 / Firul narativ: ce se discuta de fapt

La prima vedere, conversatia este despre increderea in agentii AI. In realitate, participantii delimiteaza treptat cinci probleme diferite:

1. **OBSERVABILITATE**
   Ce a executat? Cu ce date si instrumente?
2. **CORECTITUDINE**
   Cum verificam rezultatele si procesul?
3. **ARHITECTURA**
   Un agent, multi-agent sau workflow cu reguli?
4. **AUTORIZARE**
   Ce are voie sa faca si pe baza caror dovezi?
5. **EXPERIMENTE**
   Cum demonstram reproductibil ce functioneaza?

Cea mai importanta schimbare de perspectiva

> **De la „Putem avea incredere in agentii AI?” la „In ce conditii, pentru ce sarcini si pe baza caror dovezi putem permite unui sistem AI sa execute independent anumite operatiuni?”**

Prima intrebare invita la opinii despre capabilitatile modelelor. A doua poate primi raspuns prin experimente, metrici si controale tehnice.

Pozitiile participantilor

Daniel Diaconu. Pune initial accentul pe progresul modelelor si experienta practica; ulterior accepta nevoia de evaluare automatizata.

Ovidiu Boticiu. Ramane concentrat pe verificabilitate si cere sa fie demonstrat, nu presupus, avantajul multi-agent.

Sorin Balan. Conecteaza participantii, cauta un material comun si introduce perspectiva agentilor minimalisti.

Cyber AI Romania. Aduce experienta unei implementari practice cu agenti specializati si orchestrare.

Liviu Sapunaru. Separa verificarea competentei de autorizarea executiei si propune testarea erorilor de judecata.

Cri Stian. Leaga subiectele intr-o propunere evaluabila: specializare, memorie separata, evaluare progresiva, controale si benchmark.

De ce conteaza ultimele interventii ale lui Liviu

Trei agenti pot confirma acelasi document si totusi sa se insele daca toti depind de aceeasi sursa falsa. Consensul intre agenti nu echivaleaza cu independenta dovezilor. In plus, chiar daca o dovada exista, un agent poate sa o interpreteze gresit, asa cum Liviu relateaza in cazul LAB27.

In conversatie sunt descrise experimente, dar nu sunt prezentate integral baremele, metodologia sau logurile. Rezultatele relatate nu pot fi verificate independent doar pe baza acestui material.

03 / Rezumatul si ideile principale

Rezumat

Participantii incearca sa stabileasca o metoda prin care agentii AI sa poata fi evaluati, monitorizati si autorizati sa execute sarcini cu grade diferite de autonomie. Discutia porneste de la logarea executiei si verificarea rezultatelor, trece prin agenti specializati, orchestrare si memorie, apoi ajunge la separarea dintre competenta, autorizare si suficienta dovezilor.

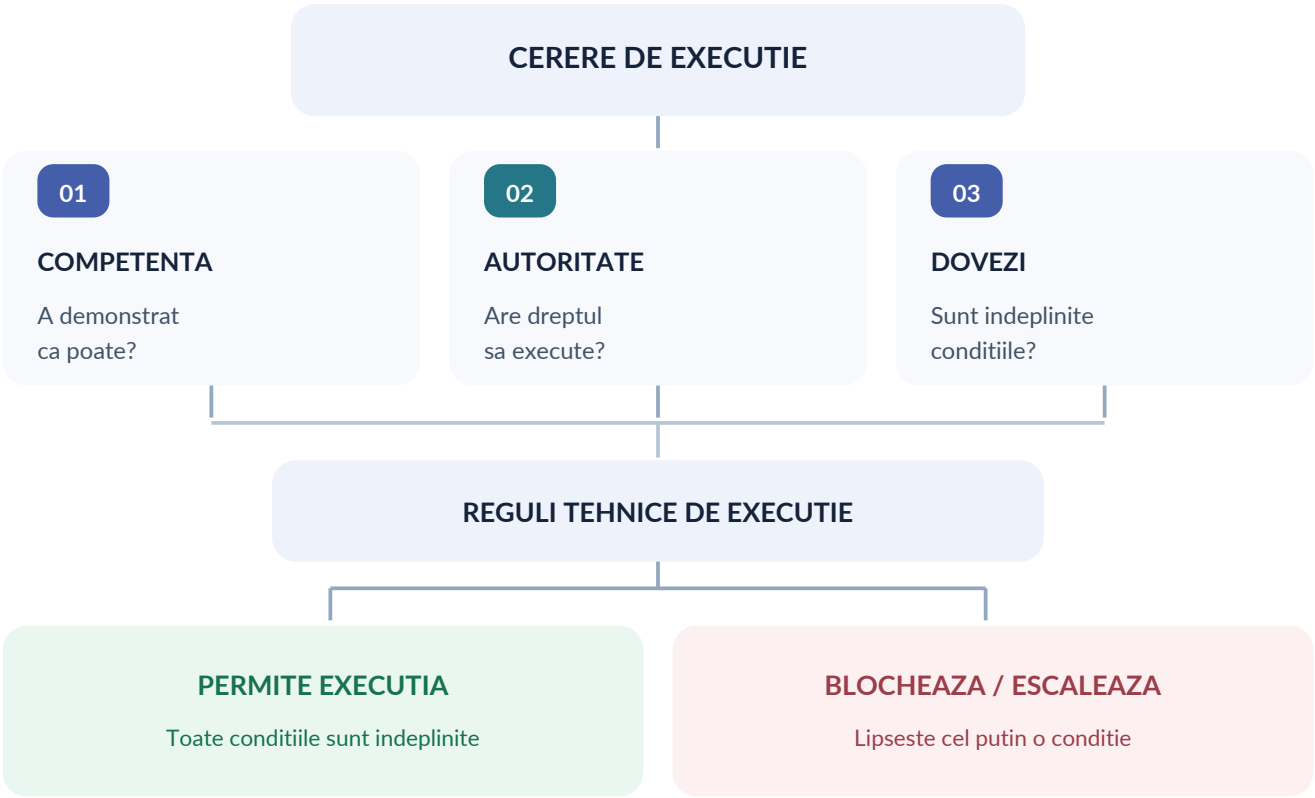
Solutia comuna care incepe sa se contureze este un **benchmark reproductibil, independent de model si framework**, care sa evalueze nu doar rezultatul final, ci si procesul, deciziile, provenienta informatiilor si capacitatea sistemului de a bloca operatiuni incorecte.

Ideile de pastrat

PRINCIPIU	SEMNIFICATIE PRACTICA
Observabilitate	Inregistrarea intrarilor, instrumentelor, argumentelor, rezultatelor si actiunilor efective.
Verificare independenta	Compararea rezultatului cu surse si criterii care nu depind exclusiv de declaratia agentului.
Evaluare progresiva	Testare controlata, apoi introducerea graduala a situatiilor dificile si neprevazute.
Trust Score	Performanta masurata pentru clase de sarcini, nu atribuita global modelului.
Autoritate separata	Performanta buna nu acorda automat permisiuni suplimentare.
Dovezi obligatorii	Executie numai dupa indeplinirea conditiilor concrete ale operatiunii.
Orchestrare evaluabila	Masurarea calitatii delegarii, a transferului de context si a deciziilor de escalare.
Memorie controlata	Provenienta, versiuni, reguli de acces si actualizare pentru informatii.
Verificarea surselor	Identificarea dependentelor dintre confirmari si a erorilor corelate.
Comparatie cu baseline	Demonstrarea beneficiilor fata de agenti simpli si workflow-uri deterministe.

Modelul rezultat din discutie

Mecanismul central poate fi reprezentat prin trei conditii obligatorii de executie: competenta, autoritate si dovezi.



Schema conceptuala a celor trei conditii propuse in schimbul de mesaje dintre Cri Stian si Liviu Sapunaru.

Ce ramane insuficient demonstrat

1. Trust Score unic. Un procent global poate ascunde erori catastrofale. Evaluarea trebuie diferentiata pe clase de operatii, severitate si risc.

2. Loguri. Arata ce s-a executat si ajuta la diagnostic, dar nu demonstreaza singure corectitudinea si nu ofera acces complet la procesele interne. Trebuie protejate daca includ date confidentiale.

3. AI doilea LLM. Nu garanteaza independenta; modelele pot repeta greseli sau pot depinde de aceeasi sursa. Pentru operatiuni critice sunt necesare reguli verificabile tehnic.

4. Arhitectura multi-agent. Nu este automat superioara. Poate adauga cost, latență, erori de transfer si noi puncte de esec.

5. Experimentele raportate. LAB27 si LAB28 reprezinta un inceput, nu o demonstratie completa. Configuratia multi-agent pe orchestrator real nu fusese testata, potrivit ultimului mesaj.

> **Directia prioritara: un set mic de teste in care comportamentul corect este stabilit anterior executiei si unde esecurile pot fi masurate reproductibil.**

04 / Raspuns propus pentru continuarea conversatiei

Varianta de publicat ca raspuns la ultimul mesaj al lui Liviu Sapunaru

Liviu Sapunaru, acum inteleg mult mai clar ce ai facut in LAB27 si LAB28. Mi se pare important ca ai separat generatorul de scenarii de evaluator si, mai ales, ca ai precizat explicit unde rezultatele experimentale nu sustin inca o concluzie. Exact genul acesta de abordare cred ca ne trebuie.

As porni chiar de la cazul concret din LAB27, in care agentul a solicitat dovada, a primit-o si totusi a concluzionat ca aceasta lipseste.

Daca poti distribui o versiune anonimizata a scenariului, baremul si traseele celor doua executii, cred ca avem deja un punct de plecare foarte bun pentru un benchmark reproductibil.

Eu as construi cateva variante controlate ale aceluiasi caz:

1. Dovada exista, lipseste sau este invalida.
2. Mai multi agenti confirma informatii provenite din aceeasi sursa, comparativ cu surse cu adevarat independente.
3. Agentul are competenta demonstrata, dar nu are autoritatea necesara.
4. Agentul are autoritate, dar lipsesc dovezile obligatorii.

Pentru fiecare scenariu stabilim inainte de executie rezultatul corect, regulile de autorizare si conditiile care trebuie sa blocheze operatiunea.

Apoi testam aceleasi cazuri pe trei configuratii: un agent simplu, un workflow cu verificari deterministe si un sistem multi-agent cu orchestrator.

Masuram nu doar rata de succes, ci si executiile nejustificate, blocarile incorecte, erorile de interpretare, costurile si capacitatea de recuperare.

Mi se pare esential sa distingem si intre verificarea unei concluzii si verificarea provenientei dovezilor. Trei agenti care repeta aceeasi informatie nu inseamna automat trei confirmari independente.

Si as merge chiar mai departe: pentru operatiunile critice, orchestratorul poate decide ce verificari solicita, dar autorizarea finala ar trebui impusa de un mecanism tehnic separat, care nu poate fi ocolit doar pentru ca agentii au ajuns la consens.

Cred ca astfel putem testa inclusiv orchestratorul, nu doar agentii individuali.

Nu avem nevoie sa construim de la inceput o infrastructura enorma. Avem nevoie de cateva cazuri bine definite, rezultate asteptate si suficiente date pentru a reproduce experimentul.

Daca putem porni de la unul dintre dosarele LAB27, cred ca am avea prima contributie concreta la benchmarkul comun despre care vorbeam. Si, foarte important, am putea descoperi inclusiv daca un sistem mai simplu rezolva problema la fel de bine.

De ce acest raspuns

Valorifica munca deja executata. Porneste de la experimentele LAB27, fara sa ceara cercetare teoretica suplimentara inaintea unui test concret.

Raspunde provocarii lui Ovidiu. Permite compararea obiectiva a orchestrarii, specializarii si verificarilor independente.

Este independent de platforma. Participantii pot executa aceleasi scenarii in propriul sistem, fara obligatia de a utiliza acelasi framework.

Concluzie

Discutia a evoluat coerent si ofera suficiente idei pentru o metodologie initiala. Nu exista insa, in materialul analizat, dovezi ca arhitectura propusa este superioara alternativelor mai simple. Cea mai valoroasa combinatie este benchmarkul propus de Cri Stian si mecanismele de evaluare decizionala prezentate de Liviu Sapunaru. **Urmatorul pas:** reproducerea unui caz LAB27, cu barem independent, in minimum doua configuratii.

ANEXA

Transcrierea integrala a conversatiei

Cele 24 de interventii sunt redate in ordinea furnizata. Au fost eliminate numai comenzile interfetei Facebook, repetitiile de preview si dublarea intrebarii initiale. Atribuirea postarii initiale lui Sorin Balan este dedusa din raspunsuri; legaturile exacte dintre toate ramurile de raspuns nu sunt disponibile.

Postarea initiala: intrebarea despre incredere si verificare

00. Sorin Balan (autorul postarii, dedus din raspunsuri)

Dragii mei, Dl. Ovidiu Boticiu ridică insistent o dilemă — sau, mai degrabă, o întrebare importantă — și cred că are dreptate. În același timp, e o chestiune la care îmi este greu să răspund. Din câte înțeleg, el întreabă cum putem verifica dacă un agent și-a făcut treaba corect. Așa-i, Ovidiu? Dacă adun frânturi din discuțiile anterioare, primul pe care l-aș ruga să revină cu lămuriri ar fi Paul Stoica, care a spus cândva că există o diferență între determinism și non-determinism [nota mea: „iarna nu-i ca vara", lol] — și că a lucra cu agenți nu e același lucru cu a scrie un program de calculator. Apoi l-aș chema la bară pe Cyber AI România, ca să ne reamintească postarea lui despre metode de „judgment". În ceea ce mă privește, am încredere în principiul *human-in-the-loop*, care presupune verificarea și aprobarea umană a acțiunilor unui agent. Cri Stian a prezentat si el, la un moment dat, un proiect în care se vedea acest principiu aplicat; dupa o actiune de agent urma o aprobare umana. Si am observat asta chiar ca tendinta generala in gandirea despre AI. Una peste alta, interfețele WebUI uzuale menționează undeva, de obicei în subsol, că „AI-ul poate greși". **Întrebarea este:** cum putem crea un material unitar care să răspundă întrebării lui Ovidiu și preocupării sale: „Putem avea încredere în agenți? - subiectul asta nu-i de nasul meu"

I. Observabilitate si verificarea rezultatelor

01. Paul Stoica

Există diferite căi de monitorizare a activității unui agent. După cum a spus și Ovidiu Boticiu, se poate monitoriza atât rezultatul, cât și modul cum a ajuns modelul la acel rezultat. Logurile complete sau măcar monitorizarea periodică sunt obligatorii.

02. Adrian Moise

Mă interesează proiectul așa că mă implic cu un comentariu . Daca greșesc ... Are cine sa mă corecteze ! Întrebarea ta , Sorin e mai profundă decât pare la prima citire. Verificarea unui agent nu seamănă cu testarea unui program clasic, pentru că nu ai o singură ieșire corectă, ai o distribuție de ieșiri posibile, iar treaba ta e să stabilești unde se termină acceptabilul și unde începe eroarea. Concret, la un agent care cheamă unelte, un log util trebuie să conțină patru lucruri, promptul exact trimis cu system promptul și contextul injectat, lista de tool calls cu argumentele exacte, răspunsurile întoarse de unelte și textul final generat. Fără primele două, nu poți distinge între o halucinație a modelului și un context prost pe care i l-ai dat tu. Un exemplu simplu, agentul meteo primește "cum e vremea la Paris", iar logul arată că a apelat get_current_weather cu location "Paris, Texas", pentru că geocoding-ul a returnat primul rezultat greșit. Rezultatul final e plauzibil, 22 de grade și senin, dar complet greșit, iar fără logul de tool call ai fi crezut că agentul funcționează. Alt exemplu, un agent de suport primește "am o problemă cu comanda", iar modelul sare direct la resolve fără să cheme check_order, deși promptul cere explicit verificarea statusului. Logul îți arată ordinea apelurilor, iar ordinea îți arată dacă modelul a respectat instrucțiunile sau doar a nimerit. Pe partea de verificare, se caută trei tipuri de semnale, tool calls cu argumente lipsă sau invalide, răspunsuri finale care nu conțin datele cerute, și pași care se repetă în buclă, de exemplu același tool call de trei ori la rând. Iar dacă vrei ceva mai solid decât human-in-the-loop, aduci un al doilea model ca verificator, cu prompt separat, care primește doar promptul, tool calls și răspunsul final, și spune dacă pașii sunt coerenti. Costul logării nu e zero, la mii de cereri pe zi logurile complete ajung la zeci de giga, așa că se păstrează trace complet câteva zile pentru debug și metrici agregate luni, altfel loghezi tot și nu citești nimic. Și un ultim punct, "AI-ul poate greși" din subsolul interfețelor nu e o scuză, e o recunoaștere că nimeni nu promite zero erori, la fel cum nici un angajat nou nu e lăsat singur pe un contract mare din prima zi.

---

03. Ovidiu Boticiu

Eu nu mă refeream la o garanție că agenții AI nu greșesc niciodată, ci la posibilitatea de a verifica ce fac efectiv și cât de corecte sunt rezultatele. Verificarea poate fi atât automată, cât și făcută de om. De exemplu, în fișierul python al agentului meteo, setarea "tool_choice: "auto"" îi permite modelului să răspundă fără să solicite instrumentul meteorologic. Dacă se afișează în terminal ce instrumente au fost apelate și ce date au returnat, se poate vedea exact ce s-a întâmplat. Apoi se poate compara răspunsul final cu datele primite și se poate verifica dacă modelul le-a redat corect.

---

04. Cri Stian

Cred ca Ovidiu Boticiu a punctat foarte bine problema. Nu este vorba despre a avea incredere oarba in agenti, ci despre posibilitatea de a verifica ce au facut efectiv si daca rezultatul este corect. As porni de la exemplul agentului meteo: ce cerere a primit, ce unealta a apelat, cu ce parametri, ce date a primit si daca raspunsul final corespunde acelor date. De aici putem construi verificari automate, teste cu rezultate cunoscute si evaluari pentru situatiile mai complexe. Human-in-the-loop ramane important, mai ales pentru actiunile cu risc, dar nu ar trebui sa inlocuiasca verificarile automate. Un alt LLM poate ajuta la evaluare, dar nici verdictul lui nu trebuie acceptat fara verificare. Ideea nu este sa eliminam complet greselile, ci sa le putem identifica, explica si corecta. Increderea in agenti trebuie construita pe rezultate verificabile, nu doar pe raspunsuri care par corecte.

II. Performanta modelelor, teste si autonomie progresiva

05. Daniel Diaconu

Aici e o intrebare cu 2 taisuri, sau mai bine zis incompleta. Suntem deja ajunsi la un nivel extrem de inalt al AI, nu vorbesc de AGI, ca e un termen putin exagerat din pct meu de vedere. O sa iau in considerare doar ideea de a executa task-uri probleme cu ajutorul ultimelor modele existente puternice, Claude Fable, GPT Astra samd, dar intr-un context nelimitat, nu printr-un abonament pro care naiba stie ce ruleaza in spate. Deci: 1. Daca ii dai task-ul de agent meteo asta e matematica pura, nimic mai mult. Am evoluat asa de mult in ultimul an incat asta o faci fara greseli cu un model simplu, local, o placa video mai modesta, nu de top, si eventual folosind un abonament Pro doar pentru verificarea rezultatelor programarii si a rularii scripturilor din spatele modelului folosit. Probabil ca vor fi greseli, insa cu siguranta asta e din cauza calitatii prompt-urilor sau a calitatii hardware-ului pe care ruleaza toata treaba. Daca ai avea acces la un modell GPT/Claude full, sunt sigur ca greselile ar fi absolut 0 la momentul actual. 2. Iesim din sfera matematicii. Daca ii dai un task extrem de complex, in care ai anumite constante, anumite variabile, deci ies la urma mai multe tip-uri de output-uri, e posibil sa iti dea si un raspuns gresit. Aici e problema de fapt de LIMITE. - Spre exemplu, incerci sa faci un studiu de piata, ai o gramada de date fixe dar si studii, statistici + il lasi pe modelul respectiv sa ia decizii. Ce sa foloseasca mai mult, ce pondere sa aplice la fiecare constanta si cum sa interpreteze variabilele. Ei, aici vei avea un alt outcome probabil cu fiecare model de AI, si e posibil ca rezultatul sa fie total pe dinafara de realitate. Aici ai LIMITA faptul ca nu ai toate datele. Dar si aici e o problema tot matematica, fara raspuns fix, deci nu poti spune ca AI-ul a gresit. - Un alt exemplu, total nematematic, il pui sa interpreteze originea universului, daca traim intr-o simulare, daca el insusi e inteligent sau nu. Modelul nu va avea toate datele, deci raspunsul lui va fi in orice caz o "interpretare inventata". Bine, limitari de fapt sunt destule in momentul de fata, pe toate modelele pro, ca altfel nu ar face fatza la atatea conturi, dar fata de luna ianuarie, de cand lucrez eu intensiv cu modele de AI, mi se pare ca diferenta e de la CER la PAMANT. Daca acum cateva luni cautam pe google/forumuri/reddit/tutoriale/youtube si cu AI, gaseam raspunsuri diverse la probleme, iar orice raspuns generat de AI il verificam (si era gresit de multe ori), acum am incredere sa las AI-ul sa imi repare serverul in productie, site-uri live, scripturi care ruleaza, programe la care lucrez. NU mi-am imaginat ca asa repede o sa avem acces la astfel de tool-uri, ma asteptam sa mai treaca vreo 3 ani, fiindca eu realizez ca milioane de joburi se vor pierde si vor fi probleme mari in economie la nivel mondial. Nu mai vorbes despre cei rau intentionati si despre utilizarea AI in razboi..

---

06. Cri Stian

Daniel Diaconu sunt de acord ca progresul din ultimele luni este enorm si ca agentii pot executa astazi sarcini care pana recent necesitau multa interventie umana. Totusi, cred ca discutia pornita de Ovidiu Boticiu este despre verificabilitate, nu neaparat despre cat de performant este modelul. Chiar si un agent meteo poate oferi un raspuns plauzibil, dar gresit: a ales alta localitate, a interpretat incorect datele sau a folosit informatii incomplete. Asta nu tine neaparat de prompt sau hardware. Nici accesul la cel mai puternic model nu garanteaza zero erori. La fel, faptul ca o problema nu are un singur raspuns corect nu inseamna ca orice rezultat este valid. Putem avea date inventate, calcule gresite sau concluzii care nu sunt sustinute de dovezi. Tocmai de aceea avem nevoie de loguri de executie, verificari automate, teste si aprobare umana acolo unde riscurile o cer. Eu folosesc agenti inclusiv pentru proiecte complexe, dar fac diferenta intre **„agentul spune ca a terminat” si „avem dovezi ca a executat corect”**. Mai ales in productie, increderea trebuie sustinuta de verificari si posibilitatea de rollback, nu doar de faptul ca pana acum lucrurile au functionat.

---

07. Daniel Diaconu

Cri Stian Exact, e extrem de important sa verifici manual, ochiometric chiar tot ce face AI-ul. Dar nu stiu daca vom mai intelege cum lucreaza peste cateva luni si daca vom avea la fel de mult control sa ii verificam efectiv executia. Nu e neaparat rau, dar nici neaparat bine :))

---

08. Cri Stian

De ce sa nu construim un mecanism de evaluare progresiva, cu verificari automate si failsafe? Initial, ii oferim agentului un mediu strict controlat: parametri exacti, actiuni permise, rezultate asteptate si criterii clare de validare. Il rulam de zeci sau sute de ori, in conditii diferite, si verificam nu doar raspunsul final, ci si ce a executat efectiv. Daca obtinem rezultate constante, ii extindem treptat libertatea de actiune. De exemplu, trecem de la o localitate prestabilita la orase alese aleatoriu, apoi la cereri ambigue, date incomplete, erori API sau intreruperi de conexiune. La fiecare etapa repetam evaluarile, inclusiv la intervale aleatorii, cu scenarii neasteptate. Verificam daca agentul identifica problemele, incearca recuperarea atunci cand este sigur sa o faca sau se opreste corect cand nu poate continua. Important este sa avem un mecanism independent care verifica rezultatele, nu doar agentul care isi declara singur succesul. Dupa suficiente teste controlate, putem elimina treptat constrangerile artificiale ale scenariilor de test si il putem lasa sa lucreze mai autonom, initial intr-un mediu izolat. Pastram insa monitorizarea, limitele de siguranta si posibilitatea de oprire. Apoi comparam rezultatele cu cele obtinute in etapa controlata. Daca, de exemplu, agentul respecta criteriile de corectitudine si executie in 90% dintre cazuri, putem incepe sa-i atribuim un anumit nivel de incredere. Evident, 90% este doar un exemplu, nu un prag universal. Conteaza si gravitatea celor 10% de esecuri. Mai mult, increderea ar trebui acordata pe categorii de sarcini, nu global. Faptul ca un agent raspunde corect la 1.000 de intrebari meteo nu inseamna automat ca il putem lasa sa administreze un server de productie. Practic, am avea un fel de Trust Score, construit din rata de succes, corectitudinea actiunilor, capacitatea de recuperare dupa erori, respectarea limitelor si stabilitatea rezultatelor in timp. Scorul ar trebui recalculat periodic si dupa modificarile importante ale modelului, promptului sau instrumentelor disponibile. Astfel, trecem de la „am incredere in agent pentru ca pare ca functioneaza” la „am un istoric verificabil care imi arata cat de bine functioneaza, in ce conditii si unde sunt limitele lui”. Cred ca asta ar raspunde mult mai concret intrebarii ridicate de Ovidiu. Nu cautam un agent care nu greseste niciodata, ci unul al carui comportament poate fi masurat, verificat si controlat. Iar autonomia lui creste pe masura ce demonstreaza ca o poate gestiona. Daniel Diaconu tocmai de aceea cred ca trebuie sa trecem de la verificarea manuala, „ochiometrica”, la mecanisme automate de evaluare si control. Nu trebuie neaparat sa intelegem fiecare decizie interna a modelului, dar trebuie sa putem verifica ce actiuni a executat, ce date a folosit si daca rezultatul respecta criteriile stabilite. De aici si ideea mea cu evaluarea progresiva: teste repetate, scenarii neprevazute, verificari independente si un nivel de incredere calculat pe baza rezultatelor reale. Pe masura ce agentul demonstreaza ca poate gestiona sarcini mai complexe, ii putem acorda mai multa autonomie, dar pastrand monitorizarea, limitele de siguranta si posibilitatea de rollback. **Pana la urma, nu trebuie sa avem control asupra fiecarui „gand” al agentului, ci asupra actiunilor si consecintelor acestora.** Altfel, increderea devine doar o presupunere. :))

---

09. Daniel Diaconu

Cri Stian Ai punctat bine de tot, mai ales la faptul ca trebuie sa ajungem la "un nivel de incredere calculat pe baza rezultatelor reale." Si la mine multe din teste au fost fix de trial and error, apoi verify error si correct error pana la urma. Eu sincer m-am aruncat cu multe in productie, din lipsa de timp. Nu ma laud ca am facut chestii cu un grad de dificultate extrem de mare, din contra, am pornit de la task-uri de baza pana am ajuns la chestii care par complicate. Am invatat si eu odata cu AI-ul, ca sa zic asa :)) Dar ai dreptate, e nevoie de un astfel de mecanism, automatizat, de evaluare progresiva. Banuiesc ca asta se foloseste de fapt si in antrenarea modelelor viitoare si ca modelele paid au toate incorporate loguri extensive cu tot ce ruleaza pe calculatoarele noastre.

III. Orchestrare, memorie si compararea arhitecturilor

10. Cri Stian

Sorin Balan, Ovidiu Boticiu, Daniel Diaconu, Paul Stoica, Adrian Moise, Cyber AI România As extinde putin discutia, dincolo de exemplul agentului meteo, la un mediu complet de lucru cu agenti AI. Personal, as merge pe **orchestrare si agenti specializati pe clase bine definite de sarcini**, nu pe un singur agent generalist care incearca sa le faca pe toate. De exemplu, un agent pentru cercetare, unul pentru implementare, altul pentru testare si unul pentru verificarea rezultatelor. Fiecare cu propriile instrumente, permisiuni, limite si criterii de evaluare. Un orchestrator ar distribui sarcinile, ar gestiona dependentele si ar decide cand un rezultat trebuie verificat, refacut sau escaladat. Avantajul nu este doar specializarea. **Reducem si riscul de poluare a contextului**, acumularea inutila de informatii si degradarea celor vechi. Fiecare agent primeste strict contextul necesar sarcinii sale, nu intreaga istorie a proiectului. Aici intervine si memoria, care trebuie tratata separat de contextul activ: ce informatii pastram, de unde provin, cat mai sunt valabile si cand trebuie actualizate sau eliminate. Iar evaluarea despre care vorbeam anterior s-ar aplica individual fiecarei clase de agenti. Nu acordam incredere intregului sistem doar pentru ca un agent a demonstrat ca se descurca bine intr-un anumit domeniu. Evident, arhitectura devine mai complexa si introduce alte posibile puncte de esec. Nu mai discutam doar despre calitatea modelului, ci si despre **orchestrare, izolare, memorie, observabilitate, evaluare si mecanisme de recuperare**. Dar tocmai aici vad eu directia: nu sa construim un agent care stie sa faca orice, ci un sistem in care agentii au responsabilitati clare, sunt evaluati separat si colaboreaza prin mecanisme verificabile. **Practic, mutam increderea de la „cat de destept este modelul?” la „cat de predictibil, verificabil si sigur este intregul sistem?”.**

---

11. Sorin Balan

Cri Stian Imaginea asta complexa de sistem pe care o descrii nu mi-e straina; mi-a stat in fata ochilor prima data cand Cyber AI România si-a descris proiectul sau. Nu mai stiu cine era atunci in grup... putini, oricum. Daca doi spuneti ca asa ar sta lucrurile corect, inseamna ca asa ar trebui sa stea.

---

12. Daniel Diaconu

Cri Stian Cu postarea asta mi-ai dat si o idee. Lucrez acum la un fel de modul ca sa ii zic asa, desi e mai mult o harababura de proiect care inseamna crearea unei MEMORII comune pentru hermes, claude, chatgpt si alti agenti din LIteLLM + inegrare n8n, cu tooluri precum forgejo, obsidian, basic memory si acontext. Dupa ce e gata si verificabil functional, o sa incerc sa atasez un layer de securitate cumva, sa vad daca cumva tocmai prin prisma memoriei/rezultatului agentilor pot sa ii faca sa se controleze unul pe altul cat de cat si sa imbunatatesc rezultatele proiectelor la care sa lucrez.

---

13. Cyber AI România

Daniel Diaconu Cri Stian cam asa este proiectul meu curent pt uz propriu...am 7 agenti si un orchestrator care deleaga.. baza este Hermes insa am scapat de dashboard ul din Hermes... mi-am creat propriul " command center". 1 agent este full stack developer, 2 cyber security 3 redactor pt siteul Cyber AI România .,4 redactor pt Diaspora International 5 postari de social media pt clientii mei 6 seo specialist 7 wordpress development si implementează ce ii zic ceilalți agentin la nevoie Hermes plus n8n plus memoria obsidian si gpt in spate pt unele lucruri... claude pt altele.. gemini pt research si viteza.. am încercat si ceva local LLM dar nu se ridica la ceea ce am eu nevoie

---

14. Daniel Diaconu

Cyber AI România Cam asa ceva incerc si eu. Nu ma deranjeaza interfata hermes, dar am probleme cu faptul ca lucrez cu 30 de proiecte si se pierde memoria, contextul, nu mai zic ca eu uit efectiv in ce punct am ajuns undeva.. Inca cochetez si cu local LLM ca nu vreau sa fiu dependent doar de paid services cum sunt acum, dar n-am ajuns cu niciun rezultat cu hardware-ul prezent acasa. Dar cat as investi in local llm + curent, mai bine dau la gepeteu :)).. Dar daca reusesc sa-mi pun panouri se schimba situatia. Ii bun ca suntem pe aceeasi lungime de unda, poate candva reusim sa facem niste tutoriale sau prezenta open source pe git moduri de lucru diverse sa invatam unii de la altii..

15. Ovidiu Boticiu

Cri Stian E interesantă arhitectura propusă, dar probabil una dintre cele mai mari dificultăți ar fi verificarea orchestratorului. Cum stabilim că distribuie corect sarcinile, transmite contextul necesar și decide corect când un rezultat trebuie verificat? Putem demonstra că un sistem multi-agent, cu orchestrator și verificare independentă, este mai fiabil decât un singur agent, folosind aceleași sarcini, resurse comparabile și criterii de evaluare? Și dacă există un avantaj, cât din el provine din specializarea agenților și cât din organizarea, controlul și verificarea procesului?

16. Sorin Balan

Ovidiu Boticiu Aici, la ce spui, poate ar incape gandirile lui Maro Zechner expuse pe blogul sau https://mariozechner.at/posts/2025-11-30-pi-coding-agent/

17. Cri Stian

Sorin Balan, Daniel Diaconu, Cyber AI România, Ovidiu Boticiu, cred ca discutia devine din ce in ce mai interesanta, mai ales ca fiecare vine cu experiente si probleme concrete. Sorin Balan, faptul ca mai multi dintre noi am ajuns la idei arhitecturale asemanatoare este incurajator, dar nu inseamna neaparat ca am gasit si solutia optima. Tocmai de aceea cred ca ar trebui sa trecem de la concepte la evaluari practice. Daniel Diaconu, problema celor 30 de proiecte este un exemplu foarte bun pentru necesitatea unui sistem de memorie bine organizat. Eu as separa memoria generala, memoria fiecarui proiect si memoria operationala a agentilor. O memorie comuna nu ar trebui sa insemne ca toti agentii citesc si modifica aceleasi informatii fara control. Avem nevoie de provenienta datelor, versiuni, reguli de actualizare si mecanisme care sa previna propagarea concluziilor gresite intre agenti. Cyber AI România, sistemul tau cu sapte agenti si un orchestrator este un exemplu concret de specializare dupa responsabilitati. Foarte interesanta si combinarea mai multor modele in functie de sarcina. As fi curios cum arata, ce face si cum gestionezi transferul contextului, conflictele dintre rezultate si verificarea executarilor, mai ales cand agentii lucreaza asupra aceluiasi proiect. Ovidiu Boticiu, intrebarile tale ating exact partea pe care consider ca trebuie sa o demonstram, nu doar sa o presupunem. **Un sistem multi-agent nu este automat mai performant sau mai fiabil decat unul cu un singur agent. El trebuie dimensionat si adaptat dupa nevoile si proiectele fiecaruia. Eu as construi un benchmark controlat, folosind aceleasi sarcini si comparand patru configuratii: 1. Un agent generalist, fara orchestrare suplimentara. 2. Acelasi agent, dar cu workflow si verificari automate. 3. Agenti specializati, coordonati prin reguli fixe. 4. Agenti specializati, cu orchestrator care decide dinamic delegarea, verificarea si escalarea. Pastram aceleasi modele, instrumente, date disponibile si resurse comparabile, pe cat posibil. Repetam testele in conditii diferite, introducem erori simulate, intreruperi, date incomplete si cereri ambigue. Masuram corectitudinea, timpul, costurile, interventiile umane, gravitatea greselilor si capacitatea de recuperare. Iar orchestratorul trebuie evaluat separat: a distribuit corect sarcina? A transmis contextul necesar? A respectat dependentele? A identificat situatiile in care trebuia sa opreasca executia sau sa solicite verificare? Putem izola experimental si contributia fiecarui mecanism. Pastram aceeasi specializare, dar schimbam orchestrarea. Apoi pastram orchestrarea si modificam verificarile. Astfel putem vedea daca imbunatatirile vin din specializarea agentilor, din controlul procesului sau din combinarea lor.** Legat de articolul lui Mario Zechner, indicat de Sorin, mi se pare un contrapunct foarte bun la ideea de a construi sisteme din ce in ce mai complexe. **Zechner explica de ce a preferat un harness minimalist pentru Pi. A renuntat la mai multe mecanisme integrate, inclusiv la suportul dedicat pentru subagenti, preferand sesiuni separate, rezultate reutilizabile si control explicit asupra contextului. Nu respinge complet subagentii. Ii considera utili in anumite situatii, precum verificarea independenta a codului, dar nu este adeptul fragmentarii inutile a activitatilor. Mi-au atras atentia si doua idei foarte practice din articol.** Prima este folosirea artefactelor persistente pentru transferul contextului. **In loc sa trimitem unui agent tot istoricul unei conversatii, putem pastra rezultatele cercetarii, planurile si deciziile in fisiere structurate, precum PLAN.md sau documente dedicate fiecarui proiect. Urmatorul agent primeste informatia relevanta, poate consulta sursele originale si continua lucrul fara sa mosteneasca toate incercarile, erorile si conversatiile anterioare. Este o abordare utila inclusiv pentru problema memoriei ridicata de Daniel.** A doua este simplitatea instrumentelor. **Pi foloseste un set de baza de numai patru instrumente pentru citire, scriere, editare si executarea comenzilor. Pentru functionalitati suplimentare, Zechner prefera adesea utilitare CLI cu documentatie citita la nevoie, in loc sa incarce contextul cu descrierile tuturor instrumentelor disponibile. Consider ca este o idee foarte buna, dar nu o poate utiliza efectiv oricine, consider ca aici trebuie putin cunostinta de mai multe chestii pentru a putea folosi un astfel de workflow. Practic, agentul afla cum sa foloseasca o unealta atunci cand are nevoie de ea, nu primeste obligatoriu manualul intregii truse la fiecare executie. Aici exista un compromis important: abordarea minimalista reduce complexitatea, dar nu rezolva automat problemele de securitate. Pi, in configuratia descrisa de Zechner, acorda implicit permisiuni foarte largi. Pentru un sistem cu acces la date sensibile sau productie, eu as pastra izolarea si controalele de executie. Din nou pentru un neexperimentat aici pot aparea probleme mari. Si tocmai aici cred ca se leaga toate ideile noastre.** Nu trebuie sa construim cel mai complex sistem posibil, ci cel mai simplu sistem care rezolva demonstrabil problema, cu un nivel acceptabil de siguranta si fiabilitate. **Daca trei clase de agenti, cu mai multe instante executate la nevoie, dau rezultate mai bune decat douazeci de roluri separate artificial, folosim trei. Daca un singur agent cu un harness bine proiectat obtine aceleasi rezultate la un cost mai mic, nu avem niciun motiv sa complicam arhitectura. Mi-ar placea sa transformam discutia intr-un proiect comun:** un set de teste, scenarii si

criterii reproductibile, independent de model sau framework, prin care fiecare sa-si poata evalua propriul sistem. **Nu neaparat sa construim toti aceeasi infrastructura, ci sa avem o metoda comuna prin care sa vedem ce functioneaza, ce nu si, mai ales, de ce. Pana la urma,** increderea intr-un sistem AI nu ar trebui sa vina din numarul agentilor sau din performanta declarata a modelului, ci din rezultatele pe care le putem verifica si reproduce.** P.S. Am postat pentru comunitate un mic "manual de termeni" https://www.facebook.com/groups/1541205557658991/permalink/1630628705383342/ Sper sa fie de folos cuiva.

IV. Competenta, mandat si conditii de executie

18. Liviu Sapunaru

Mi se pare interesantă ideea extinderii progresive a autonomiei pe baza rezultatelor obținute. Aș face însă o distincție între competența demonstrată a agentului și autoritatea pe care organizația decide să i-o acorde. Un agent poate executa impecabil mii de operațiuni, dar asta nu stabilește automat ce alte consecințe îi putem permite să producă. Rezultatele bune justifică reexaminarea mandatului, nu extinderea lui automată. Cum vedeți această separare în mecanismul de evaluare propus?

19. Cri Stian

Salutare Liviu Sapunaru, ma bucur ca te-ai alaturat. Cred ca ar trebui sa tratam competenta demonstrata si autoritatea acordata ca doua dimensiuni independente. Viziunea mea spre acest mecanism arata cam asa: 1. Competenta (Trust Score) - evaluata continuu, pe baza rezultatelor verificabile: rata de succes, tipul si gravitatea erorilor, respectarea procedurilor, comportamentul in situatii neprevazute si capacitatea de recuperare. 2. Autoritatea (Permission/Authority Level) - stabilita separat, prin politici explicite, in functie de responsabilitatile agentului, riscul operatiunilor si consecintele posibile. Aceasta nu creste automat odata cu scorul de incredere. De exemplu, un agent poate demonstra ca gestioneaza impecabil mii de operatiuni asupra unei baze de date. Asta poate justifica o propunere de extindere a responsabilitatilor, dar nu inseamna ca primeste automat dreptul de a sterge inregistrari, modifica permisiuni sau executa operatiuni ireversibile. As introduce, prin urmare, un mecanism separat de autorizare, in care rezultatele evaluarilor pot declansa o reevaluare a mandatului, dar modificarea efectiva a permisiunilor necesita aprobarea responsabilului desemnat si aplicarea unor controale tehnice. Mai mult, autonomia poate creste in interiorul mandatului existent, fara extinderea autoritatii. De exemplu, agentul poate executa independent mai multe operatiuni deja autorizate, dar ramane obligat sa solicite aprobare pentru cele cu impact ridicat. Iar procesul trebuie sa functioneze si invers: regresiile, incidentele sau schimbarile de mediu pot determina reducerea autonomiei ori suspendarea unor permisiuni. Practic, Trust Score-ul ne spune cat de bine poate executa agentul o sarcina. Mandatul stabileste ce are voie sa execute. Primul se demonstreaza prin evaluare, al doilea se acorda printr-o decizie de guvernanta, implementata prin permisiuni si limite tehnice. Cred ca aceasta separare ar trebui sa fie chiar o componenta de baza a arhitecturii propuse, nu doar o regula suplimentara.

20. Liviu Sapunaru

Cri, sunt de acord cu separarea dintre competență și autoritate. Este esențială. Eu aș introduce însă și o a treia dimensiune: suficiența probelor pentru operațiunea concretă. Un agent poate avea un Trust Score excelent și mandat valabil pentru efectuarea plăților. Dar dacă lipsește dovada recepției, plata nu trebuie executată. Nici competența demonstrată, nici autoritatea acordată nu pot înlocui acea dovadă. Aș vedea astfel trei verificări distincte: competența demonstrată pentru clasa de sarcini, autoritatea efectivă în limitele mandatului și îndeplinirea condițiilor probatorii ale operațiunii. Ultima verificare ar trebui să fie o condiție tehnică de execuție, nu doar o recomandare a unui alt agent. Și aici revin la experimentele cu mecanisme extrase din literatură. Putem folosi ChatGPT pentru a genera sistematic situații în care una dintre cele trei condiții lipsește, este simulată sau pare îndeplinită prin confirmări care provin din aceeași sursă. Apoi testăm dacă sistemul oprește operațiunea nejustificată, dar o permite pe cea pentru care toate condițiile sunt îndeplinite. Cred că aceasta ar completa foarte bine benchmarkul propus de tine.

V. Teste inspirate din literatura si validarea dovezilor

21. Liviu Sapunaru

Eu am încercat deja o abordare diferită: folosirea literaturii pentru construirea unor teste decizionale pentru agenții IA. Am pornit de la mecanisme prin care personajele ajung la convingeri greșite, confundă cauzele cu efectele sau tratează informații provenite din aceeași sursă drept confirmări independente. Am identificat 20 de pasaje literare, le-am grupat în 12 familii de mecanisme și am construit scenarii profesionale, cu metodologie, bareme, simulator și teste comparative efectiv executate. Un rezultat interesant: un agent a cerut dovada valabilității unei aprobări, a primit-o și apoi a decis că dovada lipsește. Nu am demonstrat încă superioritatea pregătirii prin literatură, dar am verificat că mecanismele literare pot fi transformate în teste concrete. Și aici văd o posibilă contribuție la proiectul discutat: să folosim literatura pentru a testa nu doar agenții, ci și sistemele care îi verifică. Putem construi, de exemplu, un caz în care trei agenți confirmă aceeași concluzie, dar toate confirmările provin dintr-o singură informație greșită. Orchestratorul trebuie să descopere problema înainte să autorizeze acțiunea. Am deja metodologia și câteva scenarii experimentale care ar putea fi adaptate în această direcție. Pentru că putem avea zece agenți, zece verificări și zece confirmări. Și o singură eroare, pe care toți o repetă.

22. Cri Stian

Liviu Sapunaru Super fain Asta chiar mi-a starnit curiozitatea maxim. Mi se pare foarte interesanta si totusi funny ideea de a transforma mecanisme narative si erori de judecata din literatura in teste concrete pentru agenti. Exemplul cu cele zece confirmari care provin, de fapt, din aceeasi sursa gresita, asta e o problema importanta pentru orice sistem multi-agent... numarul de confirmari nu inseamna neaparat independenta dovezilor. Daca toti agentii pornesc de la aceeasi informatie falsa, putem avea un consens perfect si totusi un rezultat gresit. De fapt, asta ne-ar obliga sa evaluam nu doar agentii individuali, ci si provenienta informatiei, independenta surselor si capacitatea orchestratorului de a detecta erori corelate. Mi-ar placea foarte mult sa vad mai multe detalii despre metodologia ta... cele 12 familii de mecanisme, cum ai transformat pasajele literare in scenarii profesionale... toata povestea. M-ar interesa in special cazul in care agentul a cerut dovada, a primit-o si apoi a decis ca dovada lipseste. Acolo as vrea sa vad exact traseul: ce context avea, ce a primit, ce a retinut si unde s-a produs ruptura. Cred ca materialul tau s-ar potrivi foarte bine cu ideea de benchmark si despre care discutam, asta clar, ar aduce putina diversitate fata de testele clasice. Am putea avea o categorie separata de teste pentru erori corelate si falsa confirmare independenta, nu doar pentru acuratetea raspunsului final. Daca ai metodologia documentata undeva si o poti distribui, chiar as vrea sa o citesc. Apropos... Ai testat si varianta in care agentii primesc surse deliberat separate, fata de varianta in care toti primesc acelasi context? Ar fi interesant de vazut daca diferenta dintre „consens” si „consens independent” poate fi masurata direct.

23. Liviu Sapunaru

Cri, metodologia este documentată și am deja experimente executate. Am pornit de la 20 de pasaje literare, din care am identificat 12 familii de mecanisme decizionale. Am folosit ChatGPT pentru a transpune aceste mecanisme în situații profesionale sintetice, păstrând structura cauzală, dar schimbând domeniul, personajele și contextul. Practic, ChatGPT poate funcționa ca un generator de situații de test. Îi furnizăm mecanismul, constrângerile și tipul deciziei, iar el generează scenarii, variante cu dovezi dependente sau independente, informații incomplete și consecințe diferite. Scenariile sunt apoi verificate, iar baremele sunt stabilite înainte de testare. În LAB27 am construit un simulator cu operațiuni, bugete și jurnal de execuție. Într-un caz, agentul a solicitat dovada valabilității unei aprobări, a primit-o explicit și apoi a decis că dovada lipsea. Un alt agent, cu aceleași informații, a decis corect. Avem traseul complet al ambelor execuții. Am testat și patru forme de pregătire în LAB28. Toate au rezolvat corect cele două dosare, deci nu pot afirma că pregătirea literară este superioară. Referitor la propunerea ta, putem folosi același generator pentru a construi perechi de teste multi-agent: aceeași sarcină, aceleași condiții de execuție, dar cu surse comune într-un caz și surse independente în celălalt. Orchestratorul trebuie să identifice proveniența informațiilor și să decidă dacă există suficiente dovezi pentru autorizare. Important este ca generatorul de scenarii să nu fie confundat cu evaluatorul lor. Avem nevoie de bareme independente, verificări și rezultate reproductibile. Nu am testat încă această configurație pe un orchestrator real, dar metodologia și scenariile existente oferă un punct de plecare.

Sfarsitul documentului